

Assignment #4: Matrix, 链表 & Backtracking

Updated 2226 GMT+8 Sep 29, 2025

2025 fall, Compiled by 奚兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge, Codeforces, LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

E18161: 矩阵运算

matrices, <http://cs101.openjudge.cn/pctbook/E18161/>

请使用 @ 矩阵相乘运算符。

思路：

关键步骤：

读入三个矩阵：每个矩阵先读入行数和列数，再读入矩阵元素

检查乘法条件：A的列数必须等于B的行数 (`col_a == row_b`)

计算 $A \cdot B$ ：使用 @ 矩阵相乘运算符

检查加法条件：A·B的结果必须与C的行列数相同

计算并输出结果：如果所有条件满足，计算(A·B)+C并输出

代码：

```
import numpy as np
row_a, col_a = map(int, input().split())
A = []
for i in range(row_a):
    row = list(map(int, input().split()))
    A.append(row)
A = np.array(A)

row_b, col_b = map(int, input().split())
B = []
for i in range(row_b):
    row = list(map(int, input().split()))
    B.append(row)
B = np.array(B)
row_c, col_c = map(int, input().split())
C = []
for i in range(row_c):
    row = list(map(int, input().split()))
    C.append(row)
C = np.array(C)
try:
    result = A @ B + C
    for row in result:
        print(' '.join(map(str, row)))
except ValueError:
    print("Error!")
```

代码运行截图

#50263243提交状态

[查看](#) [提交](#) [统计](#)

状态: **Accepted**

源代码

```
# 读入矩阵A
row a, col a = map(int, input().split())
```

基本信息

#: 50263243

题目: E18161

提交人: 窦兆文

E19942: 二维矩阵上的卷积运算

matrices, <http://cs101.openjudge.cn/pctbook/E19942/>

思路：

这道题要求实现二维卷积运算：

一个 $m \times n$ 的输入矩阵

一个 $p \times q$ 的卷积核

卷积核以步长1在输入矩阵上滑动

输出矩阵大小为 $(m+1-p) \times (n+1-q)$

对于输出矩阵的每个位置，我们需要：

- 1.提取输入矩阵对应位置的 $p \times q$ 子矩阵
- 2.将子矩阵与卷积核对应元素相乘
- 3.将所有乘积求和

代码：

```

m, n, p, q = map(int, input().split())
matrix = []
for i in range(m):
    row = list(map(int, input().split()))
    matrix.append(row)

kernel = []
for i in range(p):
    row = list(map(int, input().split()))
    kernel.append(row)

result = []
for i in range(m + 1 - p):
    row = []
    for j in range(n + 1 - q):
        conv_sum = 0
        for ki in range(p):
            for kj in range(q):
                conv_sum += matrix[i + ki][j + kj] * kernel[ki][kj]
        row.append(conv_sum)
    result.append(row)

for row in result:
    print(' '.join(map(str, row)))

```

代码运行截图

#50263337提交状态

[查看](#) [提交](#) [统计](#)

状态: **Accepted**

源代码

```

# 读取输入
m, n, p, q = map(int, input().split())

# 读取二维矩阵
matrix = []

```

基本信息

#: 50263337
 题目: E19942
 提交人: 窦兆文
 内存: 3656kB
 时间: 24ms
 语言: Python3

M06640: 倒排索引

data structures, <http://cs101.openjudge.cn/pctbook/M06640/>

思路:

核心思想: 倒排索引

使用字典存储倒排索引，key为单词，value为包含该单词的文档编号集合
使用集合（set）自动去重，避免同一文档编号重复记录

代码：

```
N = int(input())
inverted_index = {}
for doc_id in range(1, N + 1):
    line = input().split()
    ci = int(line[0])
    words = line[1:ci + 1]
    for word in words:
        if word not in inverted_index:
            inverted_index[word] = set()
        inverted_index[word].add(doc_id)

M = int(input())

for _ in range(M):
    query_word = input().strip()

    if query_word in inverted_index:
        doc_list = sorted(inverted_index[query_word])
        print(' '.join(map(str, doc_list)))
    else:
        print("NOT FOUND")
```

代码运行截图

#50263390提交状态

[查看](#) [提交](#) [统计](#)

状态: **Accepted**

源代码

```
# 读取文档数
N = int(input())

# 构建倒排索引
inverted_index = {}
```

基本信息

#: 50263390
题目: M06640
提交人: 窦兆文
内存: 11236kB
时间: 74ms
语言: Python3

E160.相交链表

two pinters, <https://leetcode.cn/problems/intersection-of-two-linked-lists/>

思路：

核心思想：让两个指针走相同的路径长度

指针 pA：走完链表A后，从链表B头部继续

指针 pB：走完链表B后，从链表A头部继续

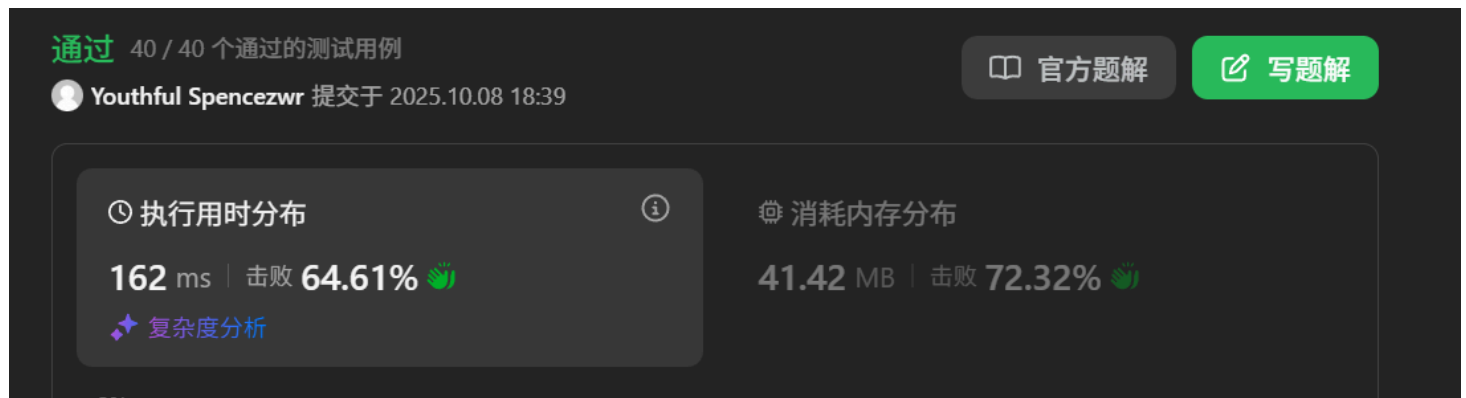
路径长度都是 $m + n$ ，所以会在相交点相遇

代码：

```
class Solution(object):
    def getIntersectionNode(self, headA, headB):
        if not headA or not headB:
            return None
        pA, pB = headA, headB
        while pA != pB:
            pA = pA.next if pA else headB
            pB = pB.next if pB else headA

        return pA
```

代码运行截图



E206.反转链表

three pointers, recursion, <https://leetcode.cn/problems/reverse-linked-list/>

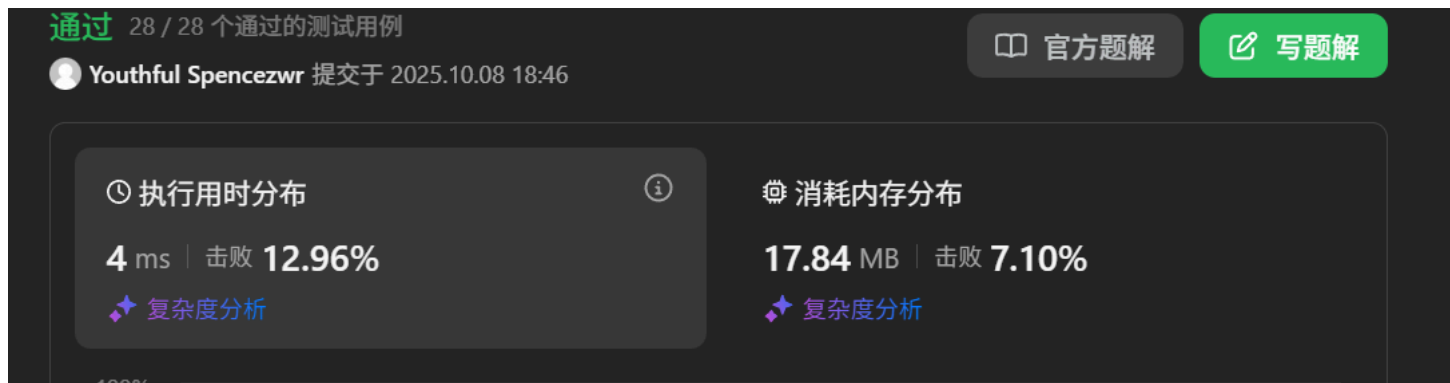
思路：

递归到链表末尾，回溯时反转指针

代码

```
class Solution(object):
    def reverseList(self, head):
        if not head or not head.next:
            return head
        new_head = self.reverseList(head.next)
        head.next.next = head
        head.next = None

        return new_head
```



T02488: A Knight's Journey

backtracking, <http://cs101.openjudge.cn/practice/02488/>

思路：

代码

(至少包含有"Accepted")

2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

本周学习对于矩阵、链表和回溯做了了解，在写矩阵运算相关题目时，很明显地感受到了python在写代码时比c++要方便的多。